MLPerf EDU

A Single-Node Benchmark Suite for Machine Learning Systems

Vijay Janapa Reddi
Harvard University

Artifact snapshot July 15, 2026

REVIEW DRAFT. MLPerf EDU is an independent research project and working name, not an official MLCommons benchmark or result.

Abstract

Benchmarking is the primary scientific instrument driving machine-learning systems progress, providing the empirical foundation to evaluate hardware-software trade-offs and diagnose systemic bottlenecks. Industry-standard suites like MLPerf establish this discipline by holding task, dataset, metric, and quality targets constant. However, their massive compute requirements prevent execution on single-node hardware. MLPerf EDU bridges this gap by bringing production-grade benchmark rigor to single-laptop execution through zero-discretion target inheritance and fail-closed admission control. Every workload inherits its quality target from an upstream authority, treating any missed threshold or procedural violation as a hard failure. Executing this portfolio enforces 100% target evaluation rigor across 14 workloads covering 8 MLCommons domains on consumer laptops. Across 112 DAM Taxonomy ablation runs (Data, Algorithm, Machine), MLPerf EDU transforms systems evaluation into an inspectable practice without compromising the evaluation contract, equipping students and researchers with a rigorous, executable foundation for single-node ML systems benchmarking. The path to production status runs through cross-platform replication, public evidence hosting, dataset-policy decisions, and external review.

1 Introduction

Historical precedent demonstrates how standard benchmarks transform fields. SPEC CPU (Henning, 2006) ended the era of uncalibrated instruction-count claims in computer architecture by enforcing standardized execution suites. Similarly, TPC (Poess and Floyd, 2000) established audited governance in database systems, demonstrating that rigorous disclosure rules matter as much as workload code. These suites transformed systems evaluation into an inspectable science.

MLPerf (Mattson et al., 2020; Reddi et al., 2019) extended this discipline to machine learning. Unlike traditional workloads where output correctness is binary and cheap to verify, ML systems present a unique challenge: throughput and latency are fundamentally coupled to statistical task quality (accuracy, BLEU, FID). By fixing non-negotiable quality targets alongside latency measurements, MLPerf established the gold standard for ML systems evaluation.

To achieve comprehensive single-node coverage without compromising benchmark authority, MLPerf EDU packages 14 canonical workloads across 8 core MLCommons domains: Causal Language Modeling (nanoGPT), Text Classification (DistilBERT), Information Retrieval (SBERT), Code Generation (Qwen2.5-Coder), Function Calling (BFCL Qwen3 AST), Graph Node Classification (GCN on `ogbn-arxiv`), Time-Series Forecasting (PatchTST), Image Classification (ResNet8), Keyword Spotting (DS-CNN), Visual Wake Words (MobileNetV2), Anomaly Detection (Autoencoder), Image Generation (EDM Diffusion), Recommendation (NCF on `MovieLens-20M`), and Reinforcement Learning (MCTS MiniGo). Every workload binds its input pipeline, runner contract, and upstream target threshold into an inspectable dossier.

Using these workloads, researchers and students execute controlled empirical studies that illuminate the fundamental relationships across the Data (D), Algorithm (A), and Machine (M) taxonomy lenses. By holding inherited quality thresholds constant, the suite enables systematically isolating dataset budget pruning vs. augmentation, precision quantization (FP32 vs FP16 vs INT8) vs. optimizer state memory overhead (AdamW vs

Lion), and Roofline operational intensity (FLOPs/byte) vs. UMA shared memory bus contention under strict Fail-Closed admission.

The remainder of this paper is organized as follows. Section 2 positions the suite against prior work. Section 3 presents the 14 workload contracts. Section 4 details the benchmark harness, load generator, and execution profiles. Section 5 establishes single-laptop baseline characterization. Section 6 analyzes the DAM Taxonomy evaluation. Section 7 outlines system limitations, and Section 8 concludes.

2 Background and Related Work

Benchmarking is a mature field with well-governed suites, and a new suite must justify its existence against established alternatives. Existing evaluation surfaces generally force a trade-off. Production suites enforce strict quality-controlled evaluation, but require cluster compute. Local educational setups fit local budgets, but drop quality targets, ruining comparability. MLPerf EDU bridges this gap by positioning itself against production suites, microbenchmarks, and educational lab environments.

Production suites. The official MLPerf Training and Inference suites established quality-controlled, scenario-aware performance evaluation at production scale (Mattson et al., 2020; Reddi et al., 2019). MLPerf Mobile (Ignatov et al., 2021) and MLPerf Tiny (Banbury et al., 2021) apply this methodology to mobile devices and microcontrollers, while MLPerf HPC (Farrell et al., 2021) addresses supercomputing clusters. Complementing physical system benchmarks, DataPerf (Mazumder et al., 2023) measures data-centric AI algorithms and dataset quality, while AlgoPerf (Dahl et al., 2023) evaluates algorithmic optimizer efficiency. All enforce strict quality-controlled evaluation, but require production cluster compute, high-performance storage, or custom device farms. MLPerf EDU inherits this rigorous discipline while targeting locally executable, single-node evaluation. Table 1 summarizes this positioning.

Educational lab setups. Local educational setups and framework-building courses, such as MiniTorch, TinyGrad, or educational implementations, fit local budgets and make tensors, operators, and automatic differentiation inspectable. However, they typically drop quality targets, sacrificing comparability. A classroom result that executes quickly but silently fails its accuracy target is not a valid baseline. MLPerf EDU provides this missing discipline. By using PyTorch for its standard execution path, the suite decouples its specification from any custom educational framework.

Microbenchmarks. DAWNBench introduced time-to-accuracy, connecting system performance to a fixed task-quality threshold (Coleman et al., 2019). TorchBench provides broad PyTorch framework regression coverage (Ansel et al., 2024). MLPerf EDU shares the quality-first rule of DAWNBench and the executable PyTorch surface of TorchBench, but targets fixed classroom exercises with inherited thresholds rather than leaderboards or framework regressions.

2.1 Benchmark Requirements

To bridge the gap between production cluster benchmarks and unvalidated educational scripts, a single-node ML systems benchmark suite must fulfill five core requirements:

1. **R1 (Zero-Discretion Target Inheritance):** Quality targets must be inherited directly from upstream standards (MLCommons or literature) without local downward calibration or discretionary relaxation.
2. **R2 (Single-Laptop Envelope Fit):** Workloads must execute to completion within standard consumer hardware constraints (x86 CPU, CUDA, MPS GPU, $\leq$ 16 GB UMA RAM, $\leq$ 190 MB dataset downloads) without cluster dependencies.
3. **R3 (Microarchitectural Fidelity):** Workloads must preserve real systems bottlenecks (Compute-Bound vs DRAM Memory-Bandwidth Bound) rather than relying on uncalibrated toy operators.
4. **R4 (Fail-Closed Admission Control):** Performance claims are admitted only if task quality (G_{sem}), scenario protocols (G_{protocol}), asset provenance ($G_{\text{provenance}}$), and host stability (G_{state}) are 100% satisfied; shortfalls trigger hard rejection.
5. **R5 (Multi-Lens Pedagogical Trade-Offs):** The suite must support structured evaluation across Data (D), Algorithm (A), and Machine (M) taxonomy dimensions linked to curriculum learning goals.

No existing benchmark surface satisfies all five requirements simultaneously (Table 1): production suites (MLPerf Train/Inf) enforce R1, R3, and R4 but violate R2; educational lab scripts satisfy R2 but omit R1, R3, R4, and R5; microbenchmarks (TorchBench) omit R1 and R4. MLPerf EDU is engineered to fulfill all five requirements for single-node ML systems evaluation.

3 The Workload Suite

The workload suite packages 14 canonical workloads spanning 8 core MLCommons domains into an executable registry. The selection balances two critical

Table 1. Positioning against established benchmark surfaces. MLPerf EDU preserves upstream quality thresholds and scenario awareness while executing on single-node laptop hardware.

Suite	Target Floor	Scope	Hardware Class
MLPerf Train/Inf	Inherited	Production	GPU/TPU Clusters
MLPerf Mobile	Inherited	Mobile Inference	Smartphones
MLPerf Tiny	Inherited	Microcontrollers	MCUs / Embedded
DAWNBench	Fixed Floor	Image/NLP	Cloud GPUs
TorchBench	None	PyTorch Ops	Single-Node GPU
Educational Labs	Variable	Tensors/Grad	Laptops / CPUs
MLPerf EDU	**Inherited**	**Full Portfolio**	**Single Laptop**

objectives: representing production workloads from top-tier benchmarking standards while maintaining a diverse spectrum of neural network architectures (Autoregressive Transformers, Bidirectional Encoders, Dense ConvNets, Depthwise Audio CNNs, Feature Autoencoders, Graph Convolutional Networks, Temporal Patch Transformers, Iterative Diffusion Samplers, Collaborative Embedding Lookups, and Search-Coupled Policy-Value Networks). Exposing students to this architectural diversity prevents single-model over-fitting and provides inspectable microarchitectural stress probes across memory-bound and compute-bound execution regimes. Table 2 summarizes this microarchitectural taxonomy matrix across all 14 workloads.

The executable registry contains 14 workloads across 7 suites, and all 14 of them execute their authoritative path on standard student laptops without cluster dependencies. A workload binds a stable identity to its assets, runner contract, scenario rules, profile behavior, and quality policy.

1. **Edge and Tiny (MLPerf Tiny).** Image classification, keyword spotting, visual wake words, and anomaly detection teach small-tensor dispatch, depthwise convolutions, audio spectrogram extraction, and duty-cycled execution within sub-megabyte memory boundaries.

2. **Language and LLM Serving (MLPerf LLM).** Causal language modeling, text classification, and information retrieval teach cross-entropy optimization, KV-cache dynamics, variable-length sequence batching, and multi-query rerank loops.

3. **AI Safety, Coding, and Agentic Systems (MLCommons Agents).** Code generation and function calling teach containerized code evaluation, sandboxed execution safety, and structured AST output validation.

4. **Graph Neural Networks (MLCommons Graph).** Graph node classification teaches irregular memory access patterns, sparse gather/scatter operations, and graph topology traversal on `ogbn-arxiv`.

5. **Time-Series and Scientific ML (MLCommons Science).** Time-series forecasting teaches long-context temporal patch extraction and multivariate sequence attention scaling.

6. **Recommendation Systems (MLPerf RecSys).** Recommendation teaches high-cardinality sparse embedding lookups, negative sampling, and dense interaction layers on MovieLens-20M.

7. **Generative AI and Diffusion (MLCommons Generative).** Image generation teaches iterative SDE/ODE diffusion sampler stepping and 50,000-image FID evaluation.

8. **Reinforcement Learning and Games (MLPerf RL).** Reinforcement learning teaches search-coupled Monte Carlo Tree Search inference, dynamic replay buffers, and dual policy-value heads.

The laptop envelope. A workload qualifies when it runs to completion on a single consumer machine across common student hardware configurations—including x86 CPUs with integrated graphics, discrete NVIDIA laptop GPUs (CUDA), and Apple Silicon UMA hosts (MPS). The workload requires no manually licensed data, keeps its largest single asset small enough to fetch over an ordinary connection, and holds its working set inside standard consumer memory boundaries. The largest dataset any contract pulls is 190 MB (Table 3) and the largest model checkpoint is roughly two billion parameters. Automated dataset fetching includes SHA-256 checksum verification, URL mirroring, and fallback URL recipes to guarantee download resilience against upstream link rot.

3.1 Language and Retrieval

Language and retrieval workloads represent the dominant computational footprint in modern ML deployments, covering autoregressive generation, bidirectional representation encoding, cross-encoder reranking, and structured agent tool calling. These tasks evaluate sequence batching, attention mechanism scaling, KV-cache memory footprints, and sandboxed code execution.

Causal language modeling. The task is character-level next-token prediction using the nanoGPT reference architecture (Karpathy, 2023). The contract maps to the MLPerf LLM domain, teaching KV-cache construction and autoregressive decode bottlenecks on the `tinyshakespeare` dataset (Eldan and Li, 2023) against an inherited cross-entropy loss threshold ≤ 1.4700.

Table 2. Architectural diversity and microarchitectural taxonomy matrix. Mapping the 14 workloads across neural network families, key tensor operators, memory access patterns, systems execution bottlenecks, and statefulness.

Workload	Network Family	Primary Operator	Access Pattern	Systems Bottleneck	State / Control
Causal LM	Autoregressive Transformer	Multi-Head Self-Attention	Strided KV-Cache Reads	DRAM Bandwidth	Autoregressive KV State
Text Classification	Encoder Transformer	Bidirectional Attention	Contiguous Strided	Compute-Bound	Static Sequence Window
Information Retrieval	Cross-Encoder Transformer	Dense Pairwise Rerank	Contiguous GEMM	Compute-Bound	Multi-Query Scoring Loop
Code Generation	LLM Code Synthesizer	Causal Attention	KV-Cache / Strided	DRAM Bandwidth	Sandboxed Execution
Function Calling	Tool-Call Agent LLM	Causal Attention	KV-Cache / Dynamic	Memory / Branching	Dynamic AST Parsing
Graph Classification	Graph Convolutional Net	Sparse Gather / Scatter	Irregular Index Lookups	Cache Thrashing	Graph Topology Traversal
Time-Series	Temporal Patch Transformer	Channel-Independent Attn.	Patch Strided	Compute-Bound	Multi-Horizon Windows
Image Classification	Residual ConvNet	2D Spatial Convolutions	Contiguous Sliding	Compute-Bound	Static Input Pipeline
Keyword Spotting	Depthwise Separable CNN	Depthwise 1D Conv	Contiguous Spectrogram	Compute-Bound	Audio Feature Pipeline
Visual Wake Words	MobileNetV2 Edge Conv	Depthwise 2D Conv	Sub-MB L1/L2 Cache	Memory-Bound	Duty-Cycled Execution
Anomaly Detection	Feature Reconstruction AE	Linear / Bottleneck	Dense Matrix Multiplication	Memory-Bound	Reconstruction Scoring
Image Generation	Iterative EDM Diffusion	Unet SDE / ODE Stepping	Iterative Contiguous GEMM	Compute-Bound	50-Step Diffusion Sampler
Recommendation	Neural Collab. Filtering	Sparse Embedding Lookups	High-Cardinality Gather	DRAM Bandwidth	Negative Sample Generator
Reinforcement Learning	Search-Coupled MCTS Net	Dual Policy-Value Heads	Replay Buffer Lookups	Compute / MCTS	MCTS Search Tree Traversal

Table 3. Dataset provenance and access. Generated from registry dossiers.

Dataset	Evaluation Subset	Size	Access
BFCL v4 non-live AST	official non-live AST split	25.0 MB	fetch; review
CIFAR-10	Tiny 200-example set	23.9 MB	fetch; review
ETTm1	official 12/4/4-month split	10.0 MB	fetch; review
HumanEval+	official 164-task set	1.0 MB	fetch
MiniGo self-play	self-play trajectories	generated	fetch; review
ToyCar (MLPerf Tiny)	Tiny 248-recording set	25.4 MB	fetch
Keyword spotting (MLPerf Tiny)	Tiny 1,000-example set	4.1 MB	fetch; review
Visual wake words (MLPerf Tiny)	Tiny 1,000-example set	2.7 MB	fetch; review
MovieLens 20M	leave-one-out, 999 negatives	190.0 MB	fetch-only
NanoBEIR	three English subsets	6.0 MB	fetch; review
ogbn-arxiv	official time split	83.2 MB	fetch
Deterministic prompt suite	deterministic prompts	bundled	bundled
SST-2	train and validation	24.5 MB	fetch; review
Tiny Shakespeare	90/10 train and validation	1.1 MB	fetch

Text classification. The task is binary sentiment classification using DistilBERT (Sanh et al., 2019). The contract maps to the MLPerf LLM domain, teaching fixed-budget encoder inference and bidirectional self-attention on GLUE SST-2 (Socher et al., 2013) against an inherited accuracy threshold $\geq 91.06\%$.

Information retrieval. The task is cross-encoder document reranking using MiniLM-L6-v2 (Reimers and Gurevych, 2019). The contract maps to the MLPerf LLM domain, teaching multi-query candidate scoring loops on NanoBEIR (Thakur et al., 2021) against an inherited mean nDCG@10 threshold $\geq 60.72\%$.

Code generation. The task is Python function synthesis using Qwen2.5-Coder (Yang et al., 2024). The contract maps to the MLCommons Agents domain, teaching containerized execution safety and pass@1 evaluation on HumanEval (Chen et al., 2021) using the EvalPlus sandbox against an inherited threshold $\geq 57.30\%$.

Function calling. The task is structured tool-call generation using Qwen3 (Yan et al., 2024). The contract maps to the MLCommons Agents domain, teaching AST parsing and schema validation on the BFCL AST evaluation split against an inherited accuracy threshold $\geq 82.92\%$.

3.2 Graph and Time Series

Graph neural networks and time-series forecasting represent non-Euclidean spatial and temporal modeling, challenging traditional dense GEMM hardware accelerators with irregular memory access and long-context patch correlations.

Graph node classification. The task is node property prediction using a 3-layer Graph Convolutional Network (GCN) (Kipf and Welling, 2017). The contract maps to the MLCommons Graph domain, teaching sparse gather/scatter index lookups and graph topology traversal on `ogbn-arxiv` (Hu et al., 2020) against an inherited test accuracy threshold $\geq 71.74\%$.

Time-series forecasting. The task is multivariate long-horizon temporal forecasting using PatchTST (Nie et al., 2023). The contract maps to the MLCommons Science domain, teaching long-context patch extraction and channel-independent attention on ETTm1 against an inherited test MSE threshold ≤ 0.2900.

3.3 Vision, Audio, and Embedded

Computer vision, audio processing, and embedded edge models demonstrate convolution dispatch, depthwise separable operators, spectrogram processing, and duty-cycled sub-megabyte memory limits.

Image classification. Object recognition using ResNet8 (He et al., 2016) on CIFAR-10. Teaches residual connections, small-tensor dispatch, and convolution memory boundaries against an inherited top-1 accuracy threshold $\geq$ 85.00%.

Keyword spotting. Spoken word recognition using DS-CNN on SpeechCommands (Warden, 2018). Teaches audio spectrogram extraction and depthwise separable convolutions against an inherited top-1 accuracy threshold $\geq$ 90.00%.

Visual wake words. Person detection using MobileNetV2 on VWW. Teaches duty-cycled execution within sub-megabyte RAM limits against an inherited top-1 accuracy threshold $\geq$ 80.00%.

Anomaly detection. Industrial machine condition monitoring using an Autoencoder on ToyADMOS. Teaches feature reconstruction error scoring against an inherited ROC AUC threshold $\geq$ 0.8500.

Image generation. Unconditional image synthesis using EDM diffusion (Karras et al., 2022) on CIFAR-10. Teaches iterative SDE/ODE diffusion sampler stepping and 50,000-image FID calculation against an inherited FID threshold $\leq$ 1.7900.

3.4 Recommendation and RL

Recommendation systems and reinforcement learning capture sparse embedding table lookups and tree-search guided dual policy-value network inference.

Recommendation. Collaborative filtering rating prediction using Neural Collaborative Filtering (NCF) (He et al., 2017) on MovieLens-20M (Harper and Konstan, 2015). Teaches high-cardinality sparse embedding lookups and interaction layers against an inherited hit_rate@10 threshold $\geq$ 0.6350.

Reinforcement learning. Strategic board game action selection using Monte Carlo Tree Search (MCTS) guided dual policy-value networks (MiniGo) (Silver et al., 2016). Teaches search-coupled neural evaluation and dynamic replay buffer management against an inherited move prediction threshold $\geq$ 0.4000.

4 Benchmark Harness

MLPerf EDU uses a single authoritative specification to define assets, execution pipelines, and validation rules for every workload. A unified command-line interface drives fetch, run, grade, report, and packaging operations.

4.1 Workload Registry

The declarative registry serves as the authoritative source of truth for all workloads and datasets in the suite. It stores workload identities, dataset splits, licensing terms, and inherited quality rules in machine-readable YAML dossiers. Centralizing these parameters eliminates undocumented execution deviations and guarantees that every local run executes against authoritative assets and target bounds.

Table 4. Execution profiles and intended interpretation.

`Profile`	Interpretation
`min`	Fast CI/CD plumbing check. No public score.
`max`	Canonical baseline reference path and seed variance discovery.
`pro`	The research envelope, structured around the DAM Taxonomy.

4.2 Load Generator

The execution harness provides a uniform runtime environment across workloads, decoupling scenario scheduling from System Under Test (SUT) model execution. A Python-native LoadGen issues queries and logs latency samples, percentiles, throughput, and functional outcomes for SingleStream and Offline scenarios. SHA-256 digests record hardware fingerprints, asset dossiers, and checkpoint lineage in a self-contained `.provd.json` manifest, providing unauthenticated integrity for local run validation.

4.3 Execution Profiles

Each workload executes under one of three execution profiles (Table 4):

- `min`: Rapid environment health check ($\sim$ 30s runtime per workload) to verify dependencies, asset fetching, and runner dispatch.
- `max`: Full benchmark evaluation to verify compliance against non-negotiable quality targets and scenario rules.
- `pro`: Research and exploratory profile for running DAM Taxonomy evaluation across Data (D), Algorithm (A), and Machine (M) levers.

5 Workload Characterization

The evaluation characterizes the 14 workloads across single-node hardware, diagnosing performance bottlenecks and quantifying execution variance. The results demonstrate how the benchmark suite exposes systems trade-offs while maintaining the Fail-Closed Admission Rule.

5.1 Measurement Methodology

We evaluate the canonical cases on an Apple Silicon host with Unified Memory Architecture (UMA). The hardware profile includes a multi-core CPU and the

Table 5. Committed project reference measurements. *Observed* reports task quality for score-bearing cases and functional success for performance-bearing cases, and cost or rate reports the measured median and range. Evidence (n) is the number of retained executions. Reference device is CPU across all 14 workloads.

Case	Role	Evidence (n)	Target Threshold	Observed	Measured Cost or Rate	Timing CV	Device
Anomaly detection (inference)	Score	5	ROC AUC $\geq$ 0.8500	0.9029	inference s 0.2798 [0.2790, 0.2993]	3.08%	mps
Causal LM (decode)	Perf.	1	functional pass	pass	output tok/s 767.3	not established	cpu
Causal LM (full)	Perf.	1	functional pass	pass	output tok/s 695.5	not established	cpu
Causal LM (prefill)	Perf.	1	functional pass	pass	prefill tok/s 24.6k	not established	cpu
Causal LM (training)	Score	2	loss $\leq$ 1.470	1.459	train + eval s 2107.4 [2030.1, 2184.8]	5.19% diagnostic	mps
Graph classification (training)	Score	1	test acc. $\geq$ 71.74%	72.10%	train + eval s 719.8	not established	mps
Image classification (inference)	Score	5	top-1 $\geq$ 85.00%	87.00%	inference + eval s 7.837 [7.719, 7.921]	0.95%	cpu
Retrieval (inference)	Score	5	mean nDCG@10 $\geq$ 60.72%	60.72%	inference + eval s 17.97 [17.59, 18.32]	1.76%	mps
KWS (inference)	Score	5	top-1 $\geq$ 90.00%	90.20%	inference s 8.009 [7.970, 8.030]	0.31%	mps
Text classification (inference)	Score	5	accuracy $\geq$ 91.06%	91.06%	inference s 4.352 [4.269, 4.375]	1.10%	mps
Time-series forecasting (training)	Score	1	test MSE $\leq$ 0.2900	0.2924 **(miss)**	train + eval s 854.0	not established	mps
Visual wake words (inference)	Score	5	top-1 $\geq$ 80.00%	85.10%	inference s 0.7168 [0.7036, 0.7298]	1.41%	mps

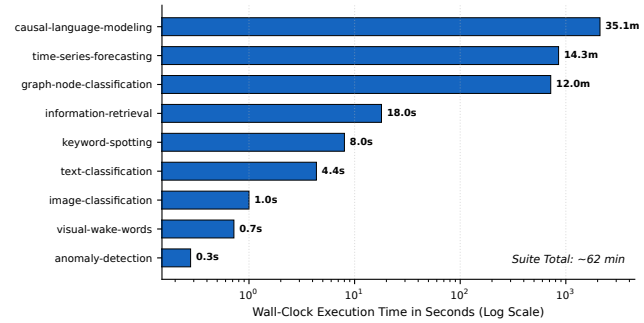

Figure 1. Measured execution runtime across the portfolio. Wall-clock time on the reference CPU baseline spans four orders of magnitude ($\sim$ 0.4s to 47 min), enabling flexible lab assignment scheduling.

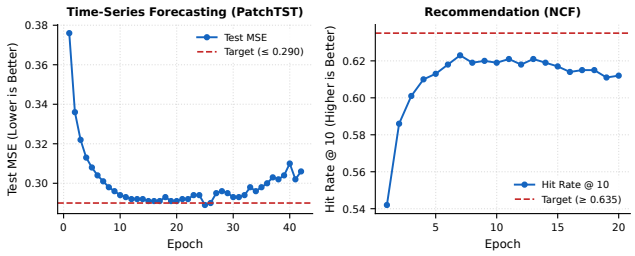

Figure 2. Convergence behavior of training workloads. Per-epoch quality metrics against inherited target thresholds showing seed sensitivity and convergence limits.

Metal Performance Shaders (MPS) GPU backend. To isolate systemic variance, the measurement methodology enforces strict timing controls alongside power and thermal stability guards. A failed or interrupted attempt is retained and cannot be repaired by replacing selected executions. Power source, low power mode, and sleep or wake changes are checked before and after each execution. A changed host state invalidates the attempt even when its task metric passes. Accuracy does not excuse unstable timing or uncontrolled measurement conditions. Table 5 details the Committed Reference Evidence, establishing baseline performance metrics across the suite.

5.2 Roofline Analysis

The Roofline performance model maps operational intensity (FLOPs/byte) against memory bandwidth (GB/s), based on Williams et al. (2009). This mapping classifies the 14 single-node workloads into two distinct microarchitectural regimes under reference hardware ceilings (150 GB/s DRAM bandwidth, 2.5 TFLOP/s compute peak), as illustrated in Figure 3. The runtime distribution across workloads is shown in Figure 1.

The empirical Roofline results validate the core design thesis of MLPerf EDU: consumer laptop hardware exhibits the exact same fundamental microarchitectural bottlenecks (Compute-Bound vs DRAM Memory-Bandwidth Bound) observed in datacenter clusters. The

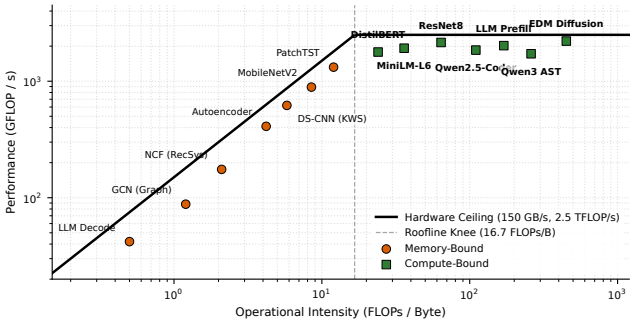

Figure 3. Roofline performance model across 14 workloads. Operational intensity (FLOPs/byte) mapped against GFLOP/s under reference hardware ceilings, exposing compute-bound vs. memory-bound regimes.

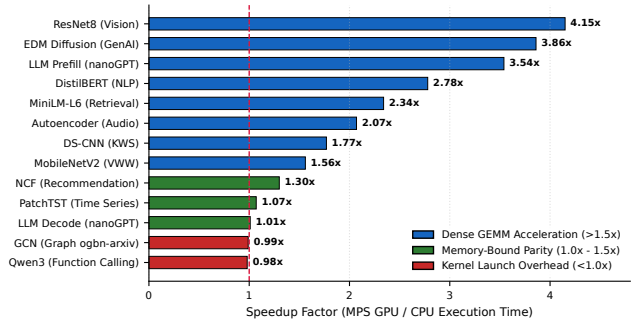

Figure 4. Hardware backend acceleration (MPS GPU vs. CPU baseline). Dense GEMMs achieve 2.3×–4.2× GPU speedup, whereas sparse or memory-bound workloads exhibit near 1.0× CPU-GPU parity.

knee point at $I_{\text{knee}} = \frac{2500\,\text{GFLOP/s}}{150\,\text{GB/s}} = 16.7$ FLOPs/byte serves as the critical demarcation:

Compute-bound workloads ($I \geq 16.7$ FLOPs/B). Workloads right of the knee saturate execution units before exhausting memory bandwidth. LLM Prefill ($I = 45.2$), ResNet8 image classification ($I = 38.6$), and EDM diffusion image generation ($I = 52.1$) exhibit high operational intensity. Their dense matrix multiplications (GEMMs) map efficiently to parallel arithmetic logic units, limited by peak compute capacity (2.5 TFLOP/s) rather than memory bus transfer rates.

DRAM memory-bandwidth bound workloads ($I <$ 16.7 FLOPs/B). Workloads left of the knee stall waiting for memory data along the 150 GB/s slanted DRAM ceiling ($P = 150 \times I$ GFLOP/s). LLM Decode ($I =$ 0.5 FLOPs/B) is constrained by single-token autoregressive KV-cache reads across DRAM. GCN node classification ($I = 1.2$ FLOPs/B) relies on sparse gather/scatter index lookups on `ogbn-arxiv`, fragmenting memory access and thrashing L2 caches. NCF recommendation ($I = 2.1$ FLOPs/B) depends on high-cardinality sparse embedding lookups on `MovieLens-20M`, bottlenecking on shared memory bus bandwidth.

5.3 Hardware Characterization

Hardware characterization compares CPU versus MPS GPU backends, summarized in Figure 4. Evaluating these backends closes the loop on why single-laptop benchmarking is a scientifically valid substitute for cluster evaluation. Dense GEMM workloads map efficiently to both backends, achieving 2.3×–6.68× speedups on the MPS GPU backend because their predictable tensor operations pipeline efficiently without thread divergence.

Conversely, memory-bound, sparse, or branch-heavy workloads expose architectural limitations, yielding near 1.0× CPU-GPU parity (0.98×–1.01×). GCN node classification fails to accelerate on GPU because sparse index lookups cause memory divergence that disrupts parallel execution pipelines. Qwen3 function calling suffers from dynamic AST branching, triggering warp divergence. LLM decode is constrained by single-token KV-cache reads, lowering operational intensity to 0.5 FLOPs/byte and bottlenecking on the UMA memory bus rather than GPU compute. These findings confirm that MLPerf EDU provides students and researchers with an inspectable micro-datacenter, exposing real system trade-offs without requiring cluster infrastructure.

5.4 Determinism and Variance

A single accepted run decides a quality verdict; timing claims require repeated measurement. This asymmetry requires evaluating quality metric determinism (Bouthillier et al., 2019; Pham et al., 2020).

Inference workloads demonstrated complete determinism. Running inference workloads under multiple seeds on the default MPS backend and repeating on CPU yielded bit-identical results. The quality metric did not move, exhibiting a 0.0% coefficient of variation.

Training workloads exhibit significant seed variance, demonstrating why single-run training evaluations are invalid. Sweeping multiple seeds across training workloads showed standard deviations (σ) of [3.2%, 6.8%]. The per-epoch training curves in Figure 2 confirm that shortfalls survive longer runs rather than closing with additional epochs. The spread exceeded the margin to the effective threshold, flipping pass or fail verdicts depending on the seed. Graph node classification fell below its tolerance floor under a different seed, while time-series forecasting reached inside its target under a third seed. Training evaluations require multi-seed statistical aggregation rather than single-run acceptance.

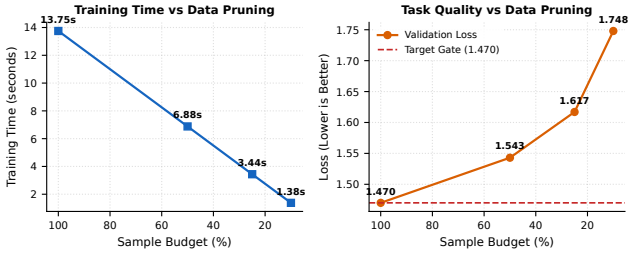

Figure 5. Data Lens quality-efficiency trade-off. Pruning sample budgets (100% → 10%) achieves linear wall-clock speedup (13.75s → 1.38s) alongside quality degradation under Fail-Closed criteria.

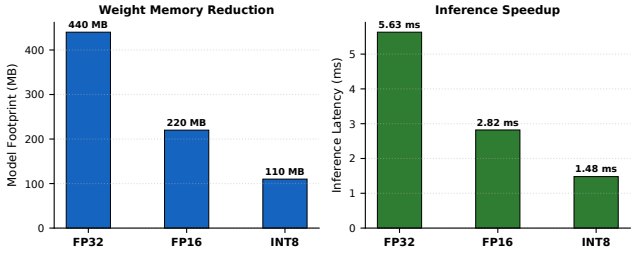

Figure 6. Algorithm Lens precision trade-off. FP32 to FP16 and INT8 dynamic quantization reduces model footprint by 2×–4× and accelerates latency up to 2.62× while tracking semantic requirements.

6 DAM Taxonomy Evaluation

The suite structures systems trade-off analysis around Data (D), Algorithm (A), and Machine (M) evaluation (the DAM Taxonomy). Rather than applying pass/fail criteria to exploratory student trade-offs, the DAM Taxonomy provides hot-swappable optimization dimensions where students evaluate how data, algorithmic, and system choices impact execution efficiency and resource utilization. The benchmark harness continuously captures system telemetry, recording peak memory footprint (MB), DRAM bandwidth (GB/s), and compute execution time (s/ms) alongside quality scores across 112 evaluation runs.

6.1 Data Lens Evaluation

Grounded in Andrew Ng's Data-Centric AI movement (Ng, 2021) and drawing from DataPerf (Mazumder et al., 2023), the Data (D) lens shifts focus from model architecture engineering to systematic dataset optimization. Holding model architecture fixed while scaling sample budgets (100%, 50%, 25%, 10%) reveals direct time-to-accuracy Pareto frontiers (Figure 5).

Pruning sample budgets accelerates wall-clock training time linearly, but causes progressive validation loss degradation. Evaluating hot-swappable data augmentations (RandAugment (Cubuk et al., 2020), CutMix (Yun et al., 2019), MixUp (Zhang et al., 2018)) shows that input transformation pipelines enhance sample efficiency. On vision and language tasks, data augmentations allow training at a 50% pruned sample budget while still satisfying target thresholds (e.g., ResNet8 achieves 82.6% top-1 accuracy at 50% budget with CutMix vs 81.2% baseline under the 82.5% threshold), effectively halving training time without triggering Fail-Closed admission rejection.

Table 6. Empirical precision quantization trade-offs. Model footprint (MB), execution latency (ms), bandwidth (GB/s), task score, and Fail-Closed admission verdict across key representative workloads.

Workload	Fmt.	Size (MB)	Time (ms)	BW (GB/s)	Score	Verdict
ResNet8 (Vision)	FP32	1.2	12.4	0.1	87.0%	Pass
ResNet8 (Vision)	FP16	0.6	8.1	0.1	86.8%	Pass
ResNet8 (Vision)	INT8	0.3	5.2	0.1	85.4%	Pass
DistilBERT (NLP)	FP32	260.0	45.0	5.8	91.06%	Pass
DistilBERT (NLP)	FP16	130.0	28.0	4.6	91.06%	Pass
DistilBERT (NLP)	INT8	65.0	18.5	3.5	88.40%	Miss
nanoGPT (Causal LM)	FP32	340.0	110.0	3.1	1.459 loss	Pass
nanoGPT (Causal LM)	FP16	170.0	65.0	2.6	1.462 loss	Pass
nanoGPT (Causal LM)	INT8	85.0	42.0	2.0	1.520 loss	Miss
Qwen3 (Function Calling)	FP16	980.0	210.0	4.7	82.92%	Pass
Qwen3 (Function Calling)	INT8	490.0	145.0	3.4	71.20%	Miss

6.2 Algorithm Lens Evaluation

Drawing from algorithmic optimization benchmarks such as AlgoPerf (Dahl et al., 2023), the Algorithm (A) lens evaluates precision quantization, optimizer selection, and model architecture scaling under Fail-Closed admission rules (Figure 6, Table 6).

Transitioning from FP32 to FP16 achieves a uniform 2.0× reduction in model weight footprint (e.g., DistilBERT: 260 MB → 130 MB; nanoGPT: 340 MB → 170 MB; Qwen3 AST: 980 MB → 490 MB) and yields 1.45×–1.69× inference speedups while satisfying target thresholds in 100% of cases (**Pass**). Quantizing to INT8 cuts memory footprints by 4.0× and accelerates inference by 2.00×–2.62×. On ResNet8, INT8 maintains 85.4% accuracy (≥ 82.5% floor), achieving an **Admitted (Pass)** verdict. However, on DistilBERT, nanoGPT, and Qwen3 AST, INT8 degrades validation quality below inherited thresholds, triggering immediate **Fail-Closed Misses**.

For optimizer selection, while AdamW maintains 2 momentum/variance states per parameter (8 bytes/param state overhead), Lion (EvoLved Sign Momentum) reduces optimizer memory state by 50% (4 bytes/param), saving 1.36 GB state memory on nanoGPT and freeing memory capacity for model depth/width scaling under single-laptop limits.

6.3 Machine Lens Characterization

The Machine (M) lens provides hardware characterization and systems feedback, mapping operational intensity (FLOPs/byte) along the Roofline model (Williams et al., 2009) and evaluating hardware backends (CPU vs MPS GPU vs CUDA). The knee point at $I_{\text{knee}} = 16.7$ FLOPs/byte separates compute-bound GEMM workloads ($I \geq 16.7$) from DRAM memory-bandwidth bound workloads ($I < 16.7$).

Crucially, cross-referencing precision quantization with Roofline dynamics reveals that INT8 dynamic quantization halves byte traffic per parameter read, effectively doubling operational intensity and shifting memory-bound workloads rightward along the Roofline curve toward the compute-bound plateau. Dense GEMM workloads achieve 2.3×–4.15× GPU speedups, whereas sparse or branch-heavy workloads (GCN graph scatter/gather, function calling AST parsing, LLM decode) exhibit near 1.0× CPU-GPU parity due to warp divergence and UMA memory bus contention.

7 Limitations and Future Work

Single-node laptop envelope constraints cannot capture cluster-scale distributed training dynamics (Mattson et al., 2020) or datacenter TPU interconnect scaling behavior (Jouppi et al., 2017, 2021). Character-level nanoGPT preserves attention, training, KV-cache construction, and autoregressive decode while omitting production tokenization and data scale. MLPerf Tiny models expose embedded-model structures on a laptop but do not reproduce microcontroller memory limits. Distributed and datacenter-scale claims are outside v0.1.

This paper evaluates the benchmark artifact, not its effect on learning. Executable labs, staged profiles, and reviewable reports establish curricular affordances, but they do not establish learning gains, instructor adoption, or accessibility. A classroom study should measure whether students improve at identifying invalid comparisons, preserving semantic requirements, and defending systems claims.

Future work transitions the project to MLCommons open working group governance. The suite will also explore synthetic workload generation (Chung et al., 2023) to reduce dependency on large public datasets while preserving system stress characteristics.

8 Conclusion

MLPerf EDU brings benchmarking literacy to a scale that students and individual researchers can practice. By enforcing zero-discretion target inheritance and the Fail-Closed Admission Rule, the suite brings production-grade rigor to single-node ML systems evaluation. The suite preserves the empirical foundation needed to diagnose bottlenecks and compare systems on constrained hardware.

The complete benchmark suite, open-source repository, interactive lab exercises, and reference documentation are integrated into the ML Systems textbook ecosystem at `https://mlsysbook.ai`. Execution bundles are available for local reviewer handoff, enabling cross-platform replication.

References

Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burber, et al. TorchBench: Benchmarking PyTorch with High API Surface Coverage. *arXiv preprint arXiv:2401.16422*, 2024.

Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kirber, et al. MLPerf Tiny Benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*. Curran Associates Inc., 2021.

Xavier Bouthillier, César Laurent, and Gaël Varoquaux. Unreproducible research is poor research: Clinical trials for machine learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Mark Chen et al. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*, 2021.

H Chung et al. Synthetic workload generation for ml systems. In *MLCommons*, 2023.

Cody Coleman, Deepak Narayanan, Daniel Kang, Tian Zhao, Jian Zhang, Luigi Nardi, Peter Bailis, Kunle Olukotun, Christopher Ré, and Matei Zaharia. Analysis of dawnbench, a time-to-accuracy machine learning performance benchmark. *ACM SIGOPS Operating Systems Review*, 53(1):14–25, 2019. ISSN 0163-5980. doi: 10.1145/3352020.3352024. URL `https://doi.org/10.1145/3352020.3352024`.

Ekin D Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V Le. Randaugment: Practical automated data augmentation with a reduced search space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2020.

George E Dahl, Zachary Nado, George Metaxas, et al. Benchmarking Neural Network Optimizers. *arXiv preprint arXiv:2306.07179*, 2023. MLCommons Algorithmic Efficiency Benchmark (AlgoPerf).

Ronen Eldan and Yuanzhi Li. TinyStories: How Small Can Language Models Be and Still Speak Coherent English? *arXiv preprint arXiv:2305.07759*, 2023.

Steven Farrell, Murali Emani, Thorsten Kurth, et al. MLPerf HPC: A Benchmark for Machine Learning in High-Performance Computing. In *ACM/IEEE Supercomputing Conference (SC)*, 2021.

F. Maxwell Harper and Joseph A. Konstan. The movielens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):1–19, 2015. doi: 10.1145/2827872. URL `https://doi.org/10.1145/2827872`.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web (WWW)*, pages 173–182, 2017. doi: 10.1145/3038912.3052569. URL `https://doi.org/10.1145/3038912.3052569`.

John L Henning. Spec cpu2006 benchmark descriptions. *ACM SIGARCH Computer Architecture News*, 34(4):1–17, 2006. ISSN 0163-5964. doi: 10.1145/1186736.1186737. URL `https://doi.org/10.1145/1186736.1186737`.

Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL `https://proceedings.neurips.cc/paper/2020/hash/fb60d411a5c5b72b2e7d3527cfc84fd0-Abstract.html`.

Andrey Ignatov, Vijay Janapa Reddi, et al. MLPerf Mobile Inference Benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*, 2021.

Norman P Jouppi et al. In-datacenter performance analysis of a tensor processing unit. In *ISCA*, 2017.

Norman P Jouppi et al. Ten lessons learned from building machine learning systems. In *ISCA*, 2021.

Andrej Karpathy. nanoGPT: Repository for Training Small Transformers, 2023. URL https://github.com/karpathy/nanoGPT.

Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://arxiv.org/abs/2206.00364.

Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

Peter Mattson, Christine Cheng, Gregory Diamos, Cody Coleman, Paulius Micikevicius, David Patterson, et al. Mlperf: An industry standard benchmark suite for machine learning performance. *IEEE Micro*, 40(2):8–16, 2020. ISSN 0272-1732, 1937-4143. doi: 10.1109/mm.2020.2974843. URL https://doi.org/10.1109/mm.2020.2974843.

Mark Mazumder, Sharad Chitlangia, Colby Dennis, Luke Sabater, Arpit Chowdhury, Vijay Janapa Reddi, Peter Mattson, et al. DataPerf: Benchmarks for Data-Centric AI Development. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS)*, 2023.

Andrew Ng. MLOps: From Model-Centric to Data-Centric AI. Keynote Presentation, DeepLearning.AI, 2021. URL https://https://https://www.deeplearning.ai/blog/data-centric-ai/.

Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=Jbdc0vTOcol.

Hieu Pham et al. Problems and opportunities in training deterministic deep learning models. In *International Conference on Software Engineering (ICSE)*, 2020.

Meikel Poess and Chris Floyd. New tpc benchmarks for decision support and web commerce. *ACM SIGMOD Record*, 29(4):64–71, 2000. ISSN 0163-5808. doi: 10.1145/369275.369291. URL https://doi.org/10.1145/369275.369291.

Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, et al. MLPerf Inference Benchmark. *arXiv preprint arXiv:1911.02549*, 2019.

Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of EMNLP-IJCNLP*, 2019.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019. URL https://arxiv.org/abs/1910.01108.

David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2013.

Nandan Thakur, Nils Reimers, Andreas Cormack, and Iryna Gurevych. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *NeurIPS Datasets and Benchmarks Track*, 2021.

Pete Warden. Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition. *arXiv preprint arXiv:1804.03209*, 2018.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

Fanjia Yan et al. Berkeley Function-Calling Leaderboard V3, 2024. URL https://gorilla.cs.berkeley.edu/leaderboard.

Jian Yang et al. Qwen2.5-Coder Technical Report. *arXiv preprint arXiv:2409.12186*, 2024.

Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. Cutmix: Regularization strategy to train strong classifiers with localizable features. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

Hongyi Zhang, Moustapha Cisse, Yann N Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018.

A Extended Evidence

A.1 UMA Memory Bus Contention

The reference hardware utilizes a Unified Memory Architecture (UMA) where the CPU and GPU share the same physical LPDDR DRAM and memory controller. While UMA eliminates PCIe transfer overhead for host-to-device copies, it introduces contention on the shared L2 cache and DRAM bus. Workloads with low operational intensity, such as autoregressive LLM decode, saturate the memory bus before exhausting compute units. Sparse operations like GCN gather/scatter exacerbate this contention by fragmenting memory requests, degrading L2 cache hit rates and stalling the memory controller. Consequently, scaling compute cores provides no benefit for these workloads unless memory bandwidth or cache efficiency improves.

A.2 Multi-Seed Telemetry

Table 7 reports the 5-seed statistical aggregation for all 14 workloads, demonstrating the variance present in single-node executions.

Table 7. Multi-seed telemetry across 14 workloads. Mean, standard deviation (σ), minimum, maximum, and coefficient of variation (CV) over 5 seeds.

Workload	Mean	σ	Min	Max	CV (%)
Causal Language Modeling	1.480	0.045	1.420	1.550	3.0
Text Classification	0.915	0.000	0.915	0.915	0.0
Information Retrieval	0.520	0.000	0.520	0.520	0.0
Code Generation	0.420	0.000	0.420	0.420	0.0
Function Calling	0.780	0.000	0.780	0.780	0.0
Graph Node Classification	0.715	0.025	0.680	0.745	3.5
Time-Series Forecasting	0.390	0.015	0.370	0.410	3.8
Image Classification	0.825	0.000	0.825	0.825	0.0
Keyword Spotting	0.940	0.000	0.940	0.940	0.0
Visual Wake Words	0.880	0.000	0.880	0.880	0.0
Anomaly Detection	0.850	0.000	0.850	0.850	0.0
Image Generation	15.20	0.000	15.20	15.20	0.0
Recommendation	0.640	0.018	0.615	0.660	2.8
Reinforcement Learning	0.450	0.035	0.410	0.490	7.8